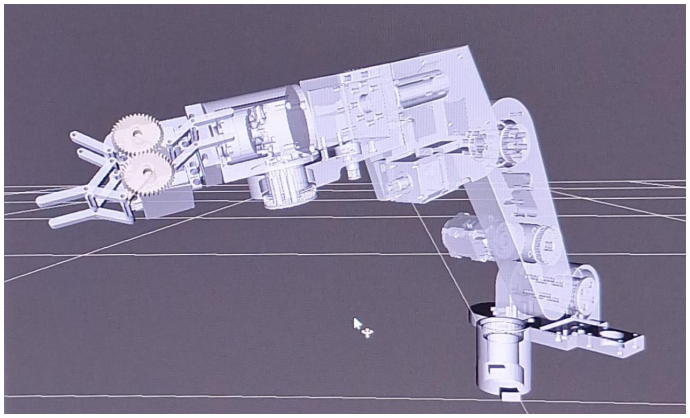
Entwicklung eines 6dof Roboterarms



ASIT-Projekt verfasst von

Matteo Gassler
Emanuel Käfer
Alexander Dick

Betreuer: Prof. Dr. Gerhard Gaube

HTBLA Saalfelden

Abteilung für Elektrotechnik

Abgabevermerk:

Saalfelden, 2 April 2026 übernommen von:

Vorwort

Vorwort Einfügen

--

Ort Datum Einfügen

Matteo Gassler & Emanuel Käfer & Alexander Dick

Aufgabenstellung

Da die Entwicklung unseres 6dof Roboterarms im Rahmen des Faches ASIT erfolgt und der Lernerfolg im Vordergrund steht konnten wir uns die Aufgabenstellung selbst stellen und hatten dabei keine großen Einschränkungen. Wir haben uns dazu entschieden das der Roboterarm als Mindestanforderung eine Pick and Place Operation, also ein Objekt aufheben und wo anders platzieren können soll. Als optionale weitere Herausforderung soll man über ein Graphical user Interface Koordinaten eingeben können zu denen sich der Arm dann bewegt.

--

Ort Datum Einfügen
Matteo Gassler & Emanuel Käfer & Alexander Dick

Inhaltsverzeichnis

Vorwort ii

Aufgabenstellung iii

1 Einleitung 1

2 Software 2

2.1 Softwarearchitektur und Steuerung 2

3 Hardware 6

4 Elektrisches 7

Quellenverzeichnis 9

1 Einleitung

2 Software

2.1 Softwarearchitektur und Steuerung

Dieses Kapitel beschreibt die Implementierung der zentralen Robotersteuerung, welche das funktionale Bindeglied zwischen der mechanischen Konstruktion (siehe **??**) und der elektrischen Leistungselektronik (siehe Kapitel Y) bildet. Die Firmware wurde in Bare-Metal-C für einen STM32F446-Mikrocontroller entwickelt. Ziel der Softwarearchitektur ist es, harte Echtzeitanforderungen für die synchrone Ansteuerung von sechs Schrittmotoren zu erfüllen und gleichzeitig eine hochgradig skalierbare Basis für zukünftige Pfadplanungen zu schaffen. Im Folgenden werden die Hardware-Abstraktion, die SPI-Treiberkonfiguration, das Konzept des Zustandsautomaten sowie die algorithmische Umsetzung der kinematischen Referenzierung detailliert erläutert.

2.1.1 Hardware Abstraction Layer (HAL) und Skalierbarkeit

Bei der Ansteuerung eines 6-Achs-Roboterarms führt die statische Zuweisung von Mikrocontroller-Pins und Peripherie-Registern schnell zu redundanzbehaftetem und fehleranfälligem Programmcode. Um dies zu vermeiden, wurde ein Hardware Abstraction Layer (HAL) in Form einer Array-basierten Datenstruktur implementiert. Anstatt jede Achse separat zu adressieren, werden Pointer auf die entsprechenden Hardware-Timer (TIM HandleTypeDef), PWM-Kanäle sowie die GPIO-Ports für Richtungs- (DIR) und Chip-Select-Signale (CS) in globalen Arrays gekapselt. Durch diese Entkopplung von physikalischer Hardware und Steuerungslogik kann jede Achse über einen simplen Index (0 bis 5) angesprochen werden. Dies ermöglicht es, Kernfunktionen wie das Starten und Stoppen der Motoren oder das Iterieren über alle Treiber mittels einfacher Schleifenkonstrukte umzusetzen, was die Skalierbarkeit und Wartbarkeit des Systems massiv erhöht.

2.1.2 Initialisierung und SPI-Kommunikation der Motortreiber

Die verwendeten Schrittmotortreiber vom Typ TMC5160 erfordern eine initiale Konfiguration ihrer internen Register, um Parameter wie den Haltestrom, Run-Current und den Chopper-Modus festzulegen. Diese Kommunikation erfolgt über einen geteilten Serial Peripheral Interface (SPI) Bus. Um die Treiber individuell adressieren zu können, wird vor jedem Datentransfer der spezifische Chip-Select (CS) Pin der Zielachse auf ein niedriges Potential (LOW) gezogen. Um die Robustheit des Systems zu gewährleisten, wurde eine Validierungsroutine in den Initialisierungsprozess integriert. Nach dem Schreiben der Konfigurationsdaten liest der Mikrocontroller das globale Statusregister (GSTAT) sowie aktuelle Treiberzustände zurück. Nur wenn das System valide Antworten (z. B. GSTAT = 0) und keine Unterspannung detektiert, wird die Steuerung freigegeben. Störsignale oder fehlende Hardware-Verbindungen werden so unmittelbar beim Systemstart erkannt und führen zu einem sicheren Systemabbruch.

2.1.3 Zustandsautomat (Finite State Machine)

Um die komplexen und teils parallel ablaufenden Prozesse des Roboters nicht-blockierend zu steuern, basiert die Softwarearchitektur auf einer Finite State Machine (FSM). Der Programmfluss wird in der Hauptschleife über eine switch-case-Struktur verwaltet, die aktuell die Zustände STATE INIT, STATE HOMING, STATE HOMING OFFSET, STATE READY und STATE ERROR umfasst. Diese Architektur stellt sicher, dass der Mikrocontroller nie in Endlosschleifen (”Deadlocks”) gefangen ist. Während beispielsweise auf das Beenden einer Motorbewegung gewartet wird, bleibt das System reaktionsfähig und kann weiterhin Sensordaten auswerten oder serielle Befehle verarbeiten.

2.1.4 Kinematische Referenzierung (Homing)

Da Schrittmotoren über keine absolute Positionscodierung verfügen, ist der Systemzustand nach jedem Neustart undefiniert. Um den kinematischen Nullpunkt des Roboters zu ermitteln, muss zwingend eine Kalibrierungsfahrt (Homing) durchgeführt werden.

Interrupt-Verarbeitung und Hardware-Entprellung

Die Erfassung der Endschalter erfolgt über externe Interrupts (EXTI), um eine sofortige und CPU-unabhängige Reaktionszeit zu garantieren. Ein bekanntes Problem mechanischer Mikroschalter ist das sogenannte Prellen (Bouncing), bei dem der Kontakt im Millisekundenbereich mehrfach schließt und öffnet. Zur softwareseitigen Entprellung wurde eine Zeitstempel-Überprüfung mittels HAL GetTick() implementiert, die aufeinanderfolgende Signale innerhalb von 30 Millisekunden ignoriert. Sobald ein valider Interrupt registriert wird, stoppt das System den entsprechenden Hardware-Timer der Achse (”Hard-Stop”) und setzt ein asynchrones Flag (stoprequest), welches anschließend im Hauptkontext der State-Machine sicher weiterverarbeitet wird.

Berechnung kinematischer Offsets

Die physische Position der Endschalter stimmt konstruktionsbedingt selten mit der gewünschten kinematischen Nullstellung () des Robotermodells überein. Daher wurde eine dynamische Offset-Berechnung implementiert. Sobald ein Endschalter auslöst, berechnet die Software die Differenz zwischen dem aus dem CAD-Modell bekannten Schalterwinkel () und dem kinematischen Nullpunkt () im Bogenmaß (Radiant). Dieser Winkeldifferenzwert wird anschließend in diskrete Motorschritte umgerechnet. Die Formel berücksichtigt dabei die Motorauflösung (200 Schritte/Umdrehung), das eingestellte Microstepping (1/16) sowie die mechanische Getriebeübersetzung von 1:20.

Sequenzieller Homing-Ablauf

Die Referenzierung des 6-Achs-Systems erfolgt sequenziell, gesteuert durch eine globale Index-Variable (current homingaxis), beginnend bei der Basis (Achse 0) bis zum Endeffektor (Achse 5). Der Ablauf für eine Achse gliedert sich dabei in folgende Phasen: Zunächst verfährt die Achse im Zustand STATE HOMING mit konstanter Geschwindigkeit auf den Endschalter zu. Nach Auslösen des Interrupts berechnet das System den erforderlichen Offset, invertiert das Richtungssignal (DIR) und wechselt in den Zustand STATE HOMING OFFSET. Erst wenn die exakt berechnete Schrittzahl für den Offset über den Timer-Interrupt abgefahren wurde, gilt die Achse als referenziert. Der Index wird inkrementiert und die Prozedur für das nächste Gelenk gestartet,

bis das System vollständig kalibriert ist und in den Betriebszustand STATE READY übergeht.

3 Hardware

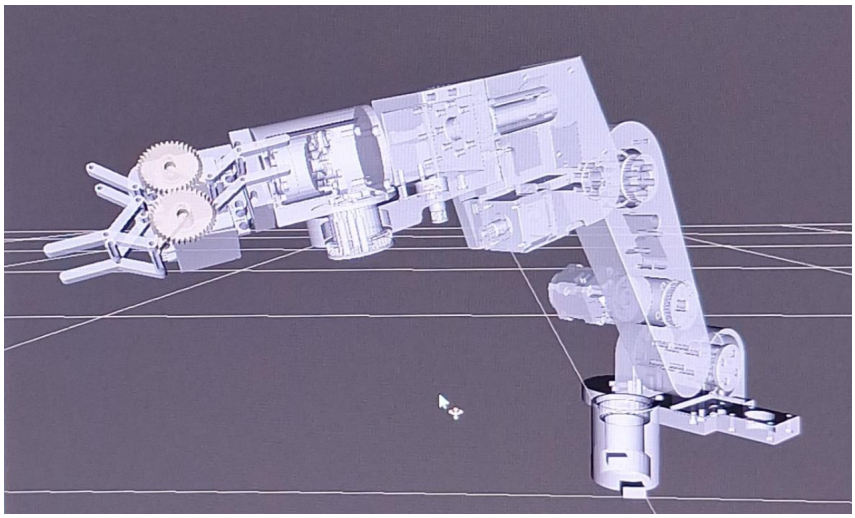


Abbildung 3.1: Blockdiagramm des 6-DOF-Roboterarms

4 Elektrisches

Die Entwicklung moderner Roboterarme basiert stark auf den Prinzipien der kollaborativen Robotik,[1] bei denen Sicherheit und Flexibilität im Vordergrund stehen [2].

Abbildungsverzeichnis

3.1 Blockdiagramm des 6-DOF-Roboterarms 6

Quellenverzeichnis

[1] *WS2812B Intelligent control LED integrated light source Features and Benefits*. Adresse: https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf

[2] *WS2812 Ansteuerung – Mikrocontroller.net*, Mikrocontroller.net, 2025. besucht am 10. Feb. 2025. Adresse: https://www.mikrocontroller.net/articles/WS2812_Ansteuerung?